

dastcore

Manual de uso

Escaner dinamico de seguridad (DAST) + asistente de bug bounty · v0.5.0

Herramienta activa e intrusiva. Uso EXCLUSIVO sobre sistemas para los que tienes autorizacion explicita y por escrito. El escaneo no arranca sin --i-have-authorization.

1. Que es dastcore

dastcore es un escaner de seguridad de aplicaciones dinamico (caja negra) escrito en Python 3.11+. Descubre la superficie, cruza cada peticion con reglas y detectores, y confirma cada hallazgo con un oraculo (diferencial, temporal, out-of-band o de reflexion) antes de reportarlo.

Diferenciadores

- Crawler dual: HTTP estatico + headless (Playwright) para SPAs y llamadas XHR/fetch.
- Descubrimiento de API por esquema: OpenAPI/Swagger + introspeccion GraphQL.
- OAST: confirmacion out-of-band de vulns ciegas (SSRF/RCE/XXE/Log4Shell) -> cero falsos positivos.
- Autorizacion multi-sesion (BOLA/IDOR/BFLA) con varias identidades/roles.
- Prueba de impacto: extrae, de forma acotada y de solo lectura, lo que un atacante obtendria.
- Cadenas de explotacion: correlaciona hallazgos en rutas de ataque con severidad compuesta.
- Capa de bug bounty: programa (scope con comodines), recon externo, hunt, triaje VRT y reportes.
- Bajo ruido: cada hallazgo pasa un oraculo de validacion antes de reportarse.

Salidas

JSON, SARIF 2.1.0 (para CI/CD), reporte HTML autocontenido, PDF, DefectDojo, y borradores Markdown de submission por plataforma de bounty.

2. Instalacion

Requiere Python 3.11+. En Windows (PowerShell), desde la raiz del repo:

```
python -m venv .venv
.venv\Scripts\pip install -e ".[dev,headless,oast,web,pdf]"

# Playwright (motor headless) - descarga Chromium:
.venv\Scripts\python -m playwright install chromium
```

Extras opcionales: headless (Playwright), oast (colaborador OOB), web (panel), pdf (fpdf2), ai (anthropic). Todos se cargan de forma perezosa: si falta uno, dastcore degrada con aviso.

Comprueba la instalacion:

```
.venv\Scripts\dastcore version
.venv\Scripts\dastcore demo # escanea un objetivo vulnerable incluido
```

3. Uso responsable: gate legal y scope

dastcore es intrusivo. Dos mecanismos, no negociables, protegen el uso:

Gate de autorizacion

Ningun escaneo, recon ni hunt arranca sin el flag --i-have-authorization. Al inicio se muestra un banner legal. Sin el flag, el comando aborta.

Scope a nivel de motor

El ScopeChecker se aplica ANTES de enviar cada peticion (y antes de guardar cada asset en recon). Deny-by-default; out-of-scope siempre gana sobre in-scope. Soporta:

- Dominios exactos (target.com, incluye el apex).
- Wildcards de subdominio (*.target.com -> subdominios, no el apex).
- Rangos CIDR (10.0.0.0/8).

```
# Ampliar / restringir scope en un scan:
dastcore scan https://staging.example.com --i-have-authorization \
  --allow-domain "*.example.com" --deny-domain admin.example.com
```

4. Comandos

Todos los comandos comparten el estilo ``dastcore <comando> [opciones]``. Usa ``dastcore <comando> --help`` para ver el detalle completo de flags.

version / demo

version imprime la version y el banner. demo levanta un objetivo vulnerable incluido y lo escanea (web + IA) para ver resultados al instante, sin objetivo propio.

```
dastcore version
dastcore demo --output demo.html
```

scan - el comando principal

Escanea un objetivo web: crawlea, ejecuta detectores activos y pasivos, y produce hallazgos confirmados por oraculo.

```
dastcore scan https://staging.example.com --i-have-authorization \
  --profile full --engine both -f sarif -o report.sarif
```

Perfiles y motor

`--profile`
quick | full | api. Fija defaults sensatos (motor, max-pages, oast).

`--engine`
http (estatico) | headless (SPA/JS) | both.

`-f / --format`
json | sarif | html | defectdojo | pdf.

`-o / --output`
Ruta del reporte (por defecto stdout; pdf requiere --output).

`--audience`
developer | executive (nivel de detalle del HTML).

Descubrimiento y ritmo

`--max-pages`
Maximo de paginas a crawlear (def. 200).

`--rps`
Peticones por segundo (def. 5).

`--concurrency`
Peticones en paralelo (def. 5).

`--max-requests`
Presupuesto total de peticones (0 = sin limite).

`--time-budget`
Presupuesto de tiempo en segundos (0 = sin limite).

`--openapi`
URL de un OpenAPI/Swagger para generar requests desde el esquema.

`--graphql`
URL de un endpoint GraphQL (introspeccion + sondas).

`--stored`
Activa deteccion de XSS/inyeccion almacenada (2 orden).

Autenticacion

`--auth-bearer`
Token Bearer (JWT/opaco).

`--auth-cookie`
Cookie de sesion (name=value).

`--auth-header`
Cabecera arbitraria (Name: value).

`--login-url / --login-field`
Form-login: URL + campos de credenciales (user=.. pass=..).

`--oauth-token-url / --oauth-client-id / ...`
OAuth2 client-credentials.

`--auth-macro / --auth-macro-var`
Login por macro (secuencia de peticiones).

`--roles-file`
JSON con identidades (name/role/auth) para BOLA/BFLA.

OAST (vulns ciegas)

Para SSRF/RCE/XXE/SSTI/CRLF/Log4shell ciegos, activa un colaborador out-of-band. Sin OAST, esos detectores son no-op (nunca inventan un hallazgo).

`--oast`
local | interactsh.

`--oast-server`
URL del servidor Interactsh propio.

Prueba de impacto (--prove-impact)

Sobre cada hallazgo YA confirmado, intenta una extraccion de solo lectura y acotada que demuestre el impacto real. Nunca crea hallazgos nuevos; si falla, el hallazgo queda intacto. Cubre:

- SQLi -> lee la version/valor de la base de datos (UNION o error-based).
- LFI -> muestra un fragmento del fichero leído (solo si casa una firma sensible).
- SSTI / code injection -> muestra la expresion evaluada.
- Command injection -> muestra la salida de `id`/`uname`.
- BOLA/IDOR -> muestra el registro ajeno accedido, redactado (emails/numeros enmascarados).

Pruebas intrusivas (tras flag, nunca en quick)

Detectores mas agresivos o con estado, desactivados por defecto:

`--waf-evasion`
Si un payload se bloquea, reintentar con tampers/encoders por familia.

`--test-race`
Condiciones de carrera (reejecuta escrituras concurrentes).

`--test-csrf`
CSRF: token no verificado.

`--test-proto-pollution`
Prototype pollution server-side (Node).

`--test-cache-poisoning`
Web cache poisoning.

`--test-weak-creds`
Credenciales por defecto/debiles (requiere --login-url).

`--test-upload`
Subida de ficheros peligrosos (sube -> recupera -> confirma).

`--test-dos`
XML entity expansion (billion laughs) + ReDoS, por diferencial temporal.

`--test-smuggling`
HTTP request smuggling (CL.TE) por diferencial temporal.

CI/CD, reanudacion y triaje

`--fail-on`

Umbral de severidad para exit != 0 (gate de CI).

`--resume`

Persiste el progreso por request; reanuda si se interrumpe.

`--baseline`

Compara contra una linea base y solo reporta lo nuevo.

`--suppress`

Aplica un .dastcore-ignore (falsos positivos aceptados).

`--config`

Fichero YAML/JSON unificado (los flags explicitos ganan).

retest / diff

retest reejecuta solo las peticiones de hallazgos previos para ver que sigue vivo y que se corrigio. diff compara dos ejecuciones y muestra nuevos/corregidos/persistentes.

```
dastcore retest prev-findings.json --i-have-authorization
dastcore diff base.json head.json
```

ai - chatbot embebido (OWASP LLM)

Escanea aplicaciones con un asistente/chatbot: autodescubre el endpoint del chat (--discover), corre prompt injection e inyeccion almacenada, y pruebas cross-tenant (BOLA/BFLA via el asistente) con una segunda identidad.

```
dastcore ai https://app.example.com --discover --i-have-authorization \
  --auth-bearer eyJ... --victim-bearer eyJ_otra_cuenta --victim-ref "unit 4B"
```

serve - panel web local

Levanta un panel local (FastAPI + SQLite) con asistente guiado de escaneo, historial y tendencias, triaje, reverificacion, diff, programados y alertas. Autocontenido (sin assets externos).

```
dastcore serve --host 127.0.0.1 --port 8000
# abre http://127.0.0.1:8000
```

baseline / cloud-serve / runner

- baseline promote: fija la linea base para CI (a partir de un escaneo).
- cloud-serve: control-plane (cola de escaneos, proyectos, tokens de runner, programados, webhooks).
- runner: agente self-hosted que toma trabajos del control-plane y ejecuta escaneos.

5. Flujo de bug bounty (programa -> recon -> hunt -> reporte)

La capa de bug bounty convierte un scope-con-comodines en una campana completa, reutilizando el motor de escaneo. Todo sigue bajo el gate --i-have-authorization y el scope a nivel de motor.

5.1 El programa (program.yaml)

Describe el scope autorizado y los limites del programa. Ejemplo minimo:

```
platform: hackerone          # hackerone|bugcrowd|intigriti|immunefi|self
handle: acme
scope:
  domains: [acme.com]
  wildcards: ['*.acme.com']  # subdominios (no el apex)
  cidrs: [203.0.113.0/24]
  out_of_scope: [blog.acme.com]
limits:
  requests_per_second: 3.0
  no_automated_scanning: false # true => solo recon/pasivo
seeds: [acme.com]            # de donde arranca el recon
payouts: { sqli: 2000, idor: 1500 } # opcional (prioriza)
```

5.2 recon - descubrir la superficie

De un scope con comodines a assets vivos e in-scope, orquestando herramientas del ecosistema en adaptadores (crt.sh, subfinder, httpx). Si una herramienta no esta instalada, se salta con aviso. Todo asset pasa por el scope ANTES de guardarse en el asset store (SQLite, con first_seen/last_seen).

```
dastcore recon --program program.yaml --i-have-authorization \
  --profile standard --db assets.db -o assets.json
```

--profile

passive (solo OSINT) | standard | deep.

--db

Asset store SQLite (attack-surface monitoring).

-o

Exporta los assets a JSON.

5.3 hunt - recon + escaneo

Descubre la superficie y escanea cada asset vivo in-scope reutilizando el pipeline de scan. Cada asset se re-verifica contra el scope antes de escanearse. Es resumible por asset (--resume). Si el programa marca no_automated_scanning, hace solo recon.

```
dastcore hunt --program program.yaml --i-have-authorization \
  --profile standard --engine http --resume hunt.json -f json -o hunt.json
```

5.4 report - borrador de submission

Genera un borrador Markdown impact-first para una plataforma, a partir de los hallazgos (salida de scan/hunt -f json). Human-in-the-loop: nunca se envia automaticamente. Incluye titulo, activo, severidad (CVSS vector + VRT + CWE), reproduccion numerada, PoC minima y no destructiva, impacto y remediacion.

```
dastcore report --input hunt.json --platform bugcrowd \
  --program program.yaml -o draft.md

# con correlacion SAST (SARIF de SastScore): sube confianza y prioridad
dastcore report --input hunt.json --sast sastscore.sarif --platform hackerone
```

--platform

hackerone | bugcrowd | generic.

--finding

ID del hallazgo (vacío = el de mayor prioridad).

--sast

SARIF de SAST; marca 'confirmado por SAST+DAST' y sube confianza.

El triaje bounty (bajo el capo del report) mapea cada hallazgo a prioridad VRT (P1-P5), deduplica reincidencias del mismo (clase + host + parametro), aplica un gate de falsos positivos y ordena por banda VRT y por impacto real (explotabilidad x payout esperado).

6. Clases de vulnerabilidad cubiertas (~62 CWE)

Cada clase se confirma con un oraculo (cero falsos positivos por diseno). Resumen por grupo:

Inyeccion (A03)

SQLi (error/boolean/time) - XSS (reflejado/DOM/almacenado) - Command injection (in-band + OAST) - NoSQL - LDAP - XPath - XML - SSTI - Code/Expression-Language injection - SSI - CRLF - HTTP response splitting - CSV/Formula - Path traversal/LFI - RFI (php wrapper).

Control de acceso y autenticacion (A01/A07)

BOLA/IDOR, BFLA, missing-auth (REST + GraphQL) - Bypass de acceso por cabeceras de confianza (X-Forwarded-For, X-Original-URL) - Session fixation - Credenciales debiles/por defecto - Mass assignment - JWT (alg:none, secreto debil, kid, confusion RS256->HS256, jku/x5u SSRF).

SSRF, XXE, deserializacion, integridad (A08/A10)

SSRF (OAST) - XXE (OAST) - Deserializacion insegura (pasiva + activa OAST) - Prototype pollution - Log4Shell/JNDI - Subida de ficheros peligrosos.

Configuracion y exposicion (A05/A02)

Cabeceras de seguridad ausentes - CORS mal configurado - Clickjacking - Cookies inseguras / SameSite=None - Reverse tabnabbing - Open redirect - Host header injection - Web cache poisoning - HTTP request smuggling - Directory listing - Metodos peligrosos - Exposicion de secretos/ficheros/source maps - Cleartext - Token de sesion en URL - Subdomain takeover - Version vulnerable conocida.

Denegacion de servicio (acotada, tras --test-dos)

XML entity expansion (billion laughs) - ReDoS (backtracking catastrofico), ambos por diferencial temporal con triple guarda anti-falsos-positivos.

IA / OWASP LLM

Prompt injection - inyeccion almacenada - fuga cross-tenant (BOLA/BFLA) via el asistente.

7. Ejemplos de flujo completo

A. Escaneo de una app conocida en CI (SARIF -> code scanning)

```
dastcore scan https://staging.example.com --i-have-authorization \
  --profile full --oast local -f sarif -o results.sarif --fail-on high
```

B. Escaneo autenticado con prueba de impacto y evasion de WAF

```
dastcore scan https://app.example.com --i-have-authorization \
  --login-url https://app.example.com/login --login-field user=admin --login-field pass=... \
  --prove-impact --waf-evasion -f html -o report.html
```

C. BOLA/BFLA con dos identidades sobre una API

```
dastcore scan https://api.example.com --i-have-authorization \
  --openapi https://api.example.com/openapi.json --roles-file roles.json
```

D. Bug bounty de punta a punta

```
dastcore recon --program program.yaml --i-have-authorization --db assets.db
dastcore hunt --program program.yaml --i-have-authorization -f json -o hunt.json
dastcore report --input hunt.json --sast sast.sarif --platform bugcrowd -o draft.md
```

8. Notas finales

- Cero falsos positivos por oraculo: si dastcore reporta algo, un oraculo lo confirmo.

- Los detectores intrusivos van tras flag y nunca en el perfil quick.
- Recon y correlacion SAST solo refuerzan; nunca inventan un hallazgo.
- Todo reporte de bounty es un borrador para tu revision; nunca hay envio automatico.
- Usa `dastcore <comando> --help` para el detalle exhaustivo de cada flag.